**TECNOLÓGICO NACIONAL DE MÉXICO**
**INSTITUTO TECNOLÓGICO DE VERACRUZ**

# REPORTE FINAL DE RESIDENCIA PROFESIONAL

**Título del Proyecto:**

Diseño e implementación de una arquitectura tecnológica basada en servicios cloud para la integración, procesamiento y visualización de datos financieros.


**Presenta:**

Nombre del Estudiante

**Asesor Interno:**

Nombre del Asesor

**Asesor Externo:**

Nombre del Asesor

Veracruz, Ver. Mayo de 2026

## Agradecimientos

Se agradece a...

## Resumen

El presente proyecto describe la implementación de una arquitectura de datos...

# Índice general

**Generalidades del Proyecto**

**1 Introducción** **1**

**2 Descripción de la empresa u organización y del puesto o área de trabajo del estudiante** **2**

2.1 Generalidades de la empresa . . . . . 2

2.2 Descripción del área de trabajo . . . . . 2

**3 Problemas a resolver** **3**

**4 Objetivos** **4**

4.1 Objetivo General . . . . . 4

4.2 Objetivos Específicos . . . . . 4

**5 Justificación** **5**

**Marco Teórico**

**6 Marco Teórico** **6**

6.1 Antecedentes . . . . . 6

6.2 Modelado Dimensional: La Fundación de la Analítica . . . . . 7
6.2.1 El Proceso de Diseño de Cuatro Pasos . . . . . 7
6.2.2 Estructuras del Esquema . . . . . 8
6.3 De Data Warehouse a Data Lakehouse: Un Cambio de Paradigma . . . . . 8
6.3.1 Del Data Warehouse al Data Lake . . . . . 9
6.4 Arquitectura Medallion: Calidad de Datos por Capas . . . . . 9
6.4.1 Capa Bronze (Raw) . . . . . 9
6.4.2 Capa Silver (Trusted) . . . . . 10
6.4.3 Capa Gold (Curated) . . . . . 10
6.5 Ingeniería de Pipelines y Orquestación con Apache Airflow . . . . . 10
6.6 Tecnologías de Procesamiento Analítico: DuckDB . . . . . 10
6.7 Pipelines Declarativos y el Rol de SQLMesh . . . . . 11
6.8 Gobierno de Datos: Marco Estratégico . . . . . 12
6.9 Sistemas ERP y el Ecosistema Odoo . . . . . 12

**Desarrollo**

**7 Procedimiento y descripción de las actividades realizadas 14**
7.1 Levantamiento de requerimientos del área de Finanzas . . . . . 14
7.1.1 Requerimientos de Información Financiera y Contable . . . . . 14
7.2 Diseño de arquitectura del sistema . . . . . 15
7.2.1 Diseño de arquitectura Data Lakehouse . . . . . 15
7.3 Creación y configuración de instancia EC2 en AWS . . . . . 15
7.3.1 Creación de EC2 . . . . . 15
7.3.2 Configuración de entorno Cloud (Linux) . . . . . 23
7.4 Creación de scripts ETL de datos con Airflow . . . . . 32
7.4.1 Scripts de Extracción a Bronce . . . . . 33
7.4.2 Ingesta de datos del Modelo de datos Bronce a Plata con SQLmesh . . 37
7.4.3 Construcción de Modelo dimensional y tablas de hechos . . . . . 40
7.5 Configuración de EC2 para exponer datos a Power BI . . . . . 43
7.5.1 Configuración de Ec2 para exposición de datos . . . . . 43
7.5.2 Configuración de Power BI service . . . . . 43

**Resultados**

**8 Resultados 45**

8.1 Dashboards de Inteligencia de Negocios . . . . . 45

**Conclusiones**

**9 Conclusiones, recomendaciones y experiencia profesional adquirida 46**
9.1 Conclusiones . . . . . 46
9.2 Recomendaciones . . . . . 47
9.3 Experiencia profesional adquirida . . . . . 48

**Competencias Desarrolladas**

**10 Competencias desarrolladas y/o aplicadas 49**

**Fuentes de Información**

**Anexos**

**A Anexos 50**
A.1 Carta de autorización de la empresa . . . . . 50

# Índice de figuras

7.1 Inicio de sesión en la consola de AWS. . . . . 16
7.2 Búsqueda del servicio EC2 en la consola de AWS. . . . . 16
7.3 Opción para lanzar una nueva instancia. . . . . 17
7.4 Asignación de nombre a la instancia EC2. . . . . 17
7.5 Selección de la imagen del sistema operativo Ubuntu. . . . . 17
7.6 Selección de la versión Ubuntu Server 24.04. . . . . 18
7.7 Selección del tipo de instancia optimizada para memoria. . . . . 18
7.8 Creación del par de claves para conexión SSH. . . . . 19
7.9 Edición de las configuraciones de red. . . . . 20
7.10 Configuración avanzada de red (VPC, subred e IP pública). . . . . 20
7.11 Configuración avanzada del volumen de almacenamiento EBS. . . . . 21

7.12 Resumen y lanzamiento de la instancia EC2. . . . . 22
7.13 Pantalla de inicio de sesión de Apache Airflow. . . . . 27
7.14 Menú principal de la consola de administración de Apache Airflow. . . . . 28
7.15 Menú para agregar una nueva conexión en Apache Airflow. . . . . 29
7.16 Vista general de los DAGs disponibles en el sistema. . . . . 30
7.17 Información general y tareas de un DAG específico. . . . . 30
7.18 Historial e información de las instancias de ejecución del DAG. . . . . 31
7.19 Vista de calendario con ejecuciones pasadas y futuras del DAG. . . . . 32

8.1 Matriz jerárquica interactiva para consolidación de gastos. . . . . 45
8.2 Panel tabular para validación de transacciones. . . . . 45
8.3 Vista de Confrontación Contable. . . . . 45

# Índice de tablas

6.1 Componentes del modelo dimensional. . . . . 8
6.2 Comparativa entre Data Warehouse Tradicional y Data Lake. . . . . 9

# Capítulo 1

# Introducción

En el entorno empresarial actual, la información financiera representa uno de los activos más críticos para la toma de decisiones estratégicas y operativas. La disponibilidad de datos confiables, consistentes y oportunos permite evaluar el desempeño organizacional, optimizar recursos y garantizar el cumplimiento de los objetivos financieros y operativos.

En este contexto, el presente proyecto tiene como finalidad el diseño e implementación de una arquitectura tecnológica orientada a la integración, procesamiento y visualización de datos financieros mediante el uso de tecnologías cloud, herramientas de ingeniería de datos y plataformas de inteligencia de negocios. El proyecto se enfoca en automatizar la extracción, transformación y carga de datos provenientes de diversas fuentes, centralizarlos en una infraestructura cloud y presentarlos mediante dashboards interactivos, lo que permite al área de Finanzas acceder a información precisa, actualizada y relevante para la toma de decisiones.

# Capítulo 2

# Descripción de la empresa u organización y del puesto o área de trabajo del estudiante

## 2.1 Generalidades de la empresa

La organización es una empresa mexicana dedicada a la comercialización de insumos agropecuarios y veterinarios, con presencia en diversas regiones del país y una amplia red de sucursales, centros de distribución y rutas de venta. La empresa tiene como misión brindar soluciones integrales al mercado agropecuario y veterinario mediante productos de calidad, precios competitivos y servicio especializado, contribuyendo al desarrollo de clientes y proveedores. Su estructura organizacional incluye áreas como Dirección General, Finanzas, Operaciones, Comercial, Sistemas y Recursos Humanos, las cuales trabajan de manera coordinada para garantizar el funcionamiento eficiente de la organización.

## 2.2 Descripción del área de trabajo

El proyecto se desarrolla en el área de Sistemas con apoyo de finanzas, quien es responsable de generar reportes financieros, consolidar información contable, analizar el desempeño financiero y apoyar la toma de decisiones estratégicas. El área requiere información confiable, oportuna y accesible, lo que hace necesaria la implementación de herramientas tecnológicas que automaticen el procesamiento y la visualización de datos.

# Capítulo 3

# Problemas a resolver

Actualmente, la organización opera con múltiples fuentes de datos distribuidas en diferentes sistemas, incluyendo el ERP, archivos Excel y otras plataformas operativas, lo que genera inconsistencias, duplicidad de información y procesos manuales en la consolidación de datos financieros.

Esta situación provoca falta de confiabilidad en la información financiera, alto consumo de tiempo en tareas manuales de conciliación y validación de datos, retrasos en la generación de reportes financieros, mayor riesgo de errores humanos y dificultad para acceder a información consolidada en tiempo real. Además, la ausencia de una arquitectura de datos centralizada dificulta la automatización de procesos y limita la capacidad del área financiera para realizar análisis eficientes y oportunos.

# Capítulo 4

# Objetivos

## 4.1 Objetivo General

Diseñar e implementar una arquitectura tecnológica basada en servicios cloud que permita integrar, procesar y visualizar los datos financieros de la organización de forma automatizada, garantizando la disponibilidad de información confiable, consistente y accesible para la toma de decisiones.

## 4.2 Objetivos Específicos

- Identificar los requerimientos del área de Finanzas para la generación de reportes.
- Diseñar la arquitectura del sistema para la integración y procesamiento de datos.
- Implementar infraestructura cloud mediante instancias EC2 en AWS.
- Desarrollar scripts ETL para automatizar la extracción, transformación y carga de datos.
- Crear dashboards interactivos en Power BI para la visualización de la información financiera.
- Integrar la infraestructura cloud con Power BI para la actualización automatizada de datos.
- Validar los resultados mediante retroalimentación del área financiera.
- Ajustar los procesos ETL y dashboards para cumplir con los requerimientos del área.

# Capítulo 5

# Justificación

La implementación de una arquitectura de datos centralizada mejora significativamente la confiabilidad, disponibilidad y calidad de la información financiera, reduce la dependencia de procesos manuales y minimiza el riesgo de errores. El proyecto aporta beneficios como la automatización de procesos de integración de datos, la reducción del tiempo de generación de reportes financieros, el incremento en la confiabilidad de la información, la mejora en la toma de decisiones estratégicas, la optimización de los recursos del área financiera y la disponibilidad de información en tiempo real mediante dashboards interactivos. Asimismo, el uso de tecnologías cloud garantiza escalabilidad, disponibilidad y seguridad en la infraestructura tecnológica.

# Capítulo 6

# Marco Teórico

## 6.1 Antecedentes

La transición estratégica hacia un modelo de inteligencia de negocio basado en Data Lake constituye un cambio de paradigma fundamental para superar las limitaciones estructurales de la infraestructura tradicional. Esta arquitectura avanzada permite alcanzar una agilidad empresarial extrema y una escalabilidad horizontal ilimitada, al tiempo que optimiza el Costo Total de Propiedad (TCO, por sus siglas en inglés) al mitigar el gasto en licenciamiento propietario y hardware especializado. La implementación de este tipo de soluciones garantiza la integridad de los activos de información mediante una gestión de datos inmutable y centralizada, abordando de manera directa los problemas críticos de fragmentación operativa y riesgo sistémico derivados de una visión empresarial descoordinada.

Uno de los principales desafíos que se resuelve mediante esta arquitectura es la eliminación de la dependencia de volcados manuales por parte de administradores de bases de datos, así como la proliferación de silos tecnológicos en hojas de cálculo aisladas. A través de la unificación de los sistemas transaccionales y analíticos, se mitigan las roturas de stock recurrentes en modelos de negocio de proximidad y se supera la incapacidad previa para realizar análisis geográficos y de segmentación local.

Desde una perspectiva técnica, el Business Intelligence (BI) transita de ser una solución de nicho, confinada a informes estáticos, a convertirse en un ecosistema abierto de Big Data que actúa como el sistema nervioso central de las organizaciones modernas. Históricamente, el BI depende de arquitecturas propietarias cerradas que fuerzan ciclos rígidos de procesamiento por lotes. En la actualidad, la competitividad se define por la agilidad en el procesamiento de volúmenes masivos de información, donde la capacidad de transformar datos brutos en decisiones ejecutivas en cuestión de minutos marca la diferencia entre el liderazgo sectorial y la obsolescencia

operativa.

Esta evolución se sintetiza en el paso de sistemas monolíticos y costosos —liderados por proveedores tradicionales— hacia plataformas basadas en estándares de código abierto. Mientras que las soluciones propietarias imponen procesos de carga restrictivos y licencias por volumen que limitan el crecimiento, las distribuciones modernas ofrecen una escalabilidad horizontal sin precedentes, permitiendo pasar de procesos de extracción lentos hacia un paradigma donde los datos fluyen en tiempo cuasi-real hacia un núcleo de almacenamiento unificado.

## 6.2 Modelado Dimensional: La Fundación de la Analítica

El modelado dimensional, conceptualizado principalmente por Ralph Kimball y Margy Ross en su obra fundamental *The Data Warehouse Toolkit*, se establece como la técnica de diseño de bases de datos de referencia para sistemas de soporte a la decisión. A diferencia del modelado de entidad-relación (ER) utilizado en sistemas transaccionales (OLTP), el cual busca eliminar la redundancia mediante la normalización, el modelado dimensional prioriza la legibilidad y el rendimiento de las consultas analíticas (OLAP).

### 6.2.1 El Proceso de Diseño de Cuatro Pasos

Kimball propone una metodología rigurosa para la construcción de modelos dimensionales:

1. **Selección del proceso de negocio:** Se identifica el evento operativo que genera datos (por ejemplo, una transacción de venta o un registro de sensor).
2. **Declaración del grano:** Es el paso más crítico; define qué representa exactamente una fila en la tabla de hechos. Un grano fino (atómico) permite mayor flexibilidad que uno agregado.
3. **Identificación de las dimensiones:** Representan el contexto del evento (“quién”, “qué”, “dónde”, “cuándo”).
4. **Identificación de los hechos:** Son las métricas cuantitativas que resultan del proceso de negocio.

### 6.2.2 Estructuras del Esquema

El modelo resultante suele adoptar la forma de un **Esquema de Estrella**, donde una tabla de hechos central está rodeada por tablas de dimensiones desnormalizadas. El **Esquema de Copo de Nieve** es una variación donde las dimensiones se normalizan parcialmente, aunque suele evitarse en entornos analíticos puros por la complejidad que añade a las consultas SQL.

**Tabla 6.1.** Componentes del modelo dimensional.

| Componente | Características Técnicas | Función Analítica |
|---|---|---|
| Tablas de Hechos | Contienen claves foráneas (FK) y medidas numéricas. | Soportar agregaciones (SUM, AVG) y cálculos de KPI. |
| Tablas de Dimensiones | Contienen atributos descriptivos (texto) y claves subrogadas. | Proporcionar filtros, agrupaciones y etiquetas para reportes. |

## 6.3 De Data Warehouse a Data Lakehouse: Un Cambio de Paradigma

La arquitectura de datos evoluciona significativamente, pasando del **Data Warehouse (DW)** tradicional, que resulta rígido ante datos no estructurados y costoso de escalar, al **Data Lake**, que ofrece almacenamiento masivo y económico pero que a menudo se degrada en un "pantano de datos" por su falta de esquemas y transaccionalidad.

El **Data Lakehouse** surge para resolver estas deficiencias, combinando las funciones de gestión y rendimiento del DW e implementándolas directamente sobre el almacenamiento de objetos del Data Lake. Los pilares fundamentales de esta arquitectura incluyen:

- **Transacciones ACID:** Garantizan la integridad de los datos en escrituras concurrentes.
- **Esquemas Evolutivos:** Permiten la modificación del esquema sin interrupción del servicio.
- **Soporte Nativo para IA y ML:** Permiten el acceso directo a los archivos para algoritmos de ciencia de datos sin capas SQL intermedias.

### 6.3.1 Del Data Warehouse al Data Lake

La transición del Data Warehouse tradicional al Data Lake no es solo una actualización de software, sino un cambio de paradigma hacia la agilidad empresarial extrema. Mientras el DW exige que el dato se adapte a un molde predefinido antes de guardarse, el Data Lake permite que la empresa trabaje con sus datos en formato nativo, postergando la estructura hasta el momento preciso del consumo.

**Tabla 6.2.** Comparativa entre Data Warehouse Tradicional y Data Lake.

| **Característica** | **Data Warehouse Tradicional** | **Data Lake** |
|---|---|---|
| Paradigma | Schema-on-Write (rígido) | Schema-on-Read (flexible) |
| Ingesta de datos | Lenta (requiere transformación previa) | Inmediata (formato nativo) |
| Gobernanza/Latencia | Alta latencia de actualización | Baja latencia; acceso inmediato |
| Escalabilidad | Vertical y costosa (Vendor Lock-in) | Horizontal e ilimitada |
| Costo Total (TCO) | Elevado (licenciamiento por TB) | Optimizado (código abierto) |

## 6.4 Arquitectura Medallion: Calidad de Datos por Capas

En el contexto de los Data Lakehouses modernos, la arquitectura Medallion (o arquitectura de capas de calidad) proporciona un flujo lógico para el refinamiento de la información. Este enfoque permite gestionar la "fuente de verdad" de manera incremental.

### 6.4.1 Capa Bronze (Raw)

Es el punto de entrada. Los datos se almacenan en su estado nativo, tal como provienen de la fuente (ERP, APIs, IoT). No se aplican transformaciones, lo que permite que esta capa actúe como un registro histórico inmutable para futuros reprocesos.

### 6.4.2 Capa Silver (Trusted)

En esta etapa se aplican procesos de limpieza, filtrado y normalización básica. Los datos se transforman a formatos optimizados (como Parquet o Delta). Es la capa donde se resuelven las inconsistencias de tipos de datos y se aplican reglas de validación iniciales.

### 6.4.3 Capa Gold (Curated)

Representa el nivel más alto de refinamiento. Los datos se organizan siguiendo modelos dimensionales (Kimball) o estructuras agregadas específicas para casos de uso de negocio, como cuadros de mando integrales o modelos de aprendizaje automático.

## 6.5 Ingeniería de Pipelines y Orquestación con Apache Airflow

Un **Pipeline de Datos** constituye la infraestructura esencial para mover información desde un sistema de origen hacia un destino, aplicando transformaciones críticas en el proceso. Dada la creciente complejidad de estos flujos, la orquestación se vuelve indispensable.

Apache Airflow, siguiendo la filosofía "Workflow as Code", aborda esta necesidad al permitir la definición de flujos de trabajo mediante código Python, lo que facilita el uso de control de versiones, pruebas unitarias y la colaboración entre ingenieros. La representación lógica de este pipeline se materializa en un **Grafo Acíclico Dirigido (DAG)**, cuyas propiedades fundamentales son:

- **Dirigido:** Establece un orden claro de ejecución de las tareas.
- **Acíclico:** Asegura que el flujo avance siempre hacia un final sin bucles infinitos.
- **Idempotente:** Garantiza que los mismos parámetros produzcan siempre el mismo resultado, facilitando la recuperación ante fallos.

## 6.6 Tecnologías de Procesamiento Analítico: DuckDB

DuckDB representa una innovación en el ámbito de las bases de datos analíticas embebidas. Se destaca su capacidad para realizar procesamiento OLAP directamente en el entorno de ejecución del usuario (*in-process*), eliminando la latencia de red de los sistemas cliente-servidor tradicionales.

A diferencia de los motores de bases de datos tradicionales que procesan una fila a la vez, DuckDB utiliza **ejecución vectorizada**, procesando grandes bloques de datos simultáneamente. Esta técnica optimiza el uso de la caché de la CPU y permite un rendimiento superior en consultas complejas sobre grandes volúmenes de datos.

## 6.7 Pipelines Declarativos y el Rol de SQLMesh

La ingeniería de datos experimenta una transición de pipelines imperativos, donde cada paso debe programarse explícitamente, hacia **pipelines declarativos**, en los cuales el ingeniero describe el estado final deseado de los datos y el sistema subyacente gestiona su ejecución óptima.

En este contexto, SQLMesh emerge como una solución fundamental que integra conceptos avanzados de desarrollo de software para la gestión del ciclo de vida de los datos. Sus funcionalidades clave incluyen:

- **Virtual Data Environments:** Facilitan la creación de entornos aislados para pruebas sin la duplicación física de datos.
- **Planificación Automática:** Identifica inteligentemente qué modelos deben ser recalculados tras cambios en el código para optimizar los costos computacionales.
- **Auditoría Integrada:** Permite el seguimiento detallado de las modificaciones en la estructura de los datos.

## 6.8 Gobierno de Datos: Marco Estratégico

El gobierno de datos (*Data Governance*) se define como el ejercicio de la toma de decisiones y la autoridad sobre los activos de datos. Sus componentes clave incluyen:

- **Políticas y Estándares:** Reglas sobre cómo se definen y utilizan los datos.
- **Data Stewardship:** Roles responsables de la calidad y el significado de los datos en áreas de negocio específicas.
- **Catálogo de Datos:** Herramienta técnica que documenta el linaje, la definición y el uso de los datos en la organización.

Además, la gobernanza se apoya en roles fundamentales:

- **Data Owner (Dueño de los Datos):** Tiene la máxima responsabilidad sobre la calidad, seguridad y cumplimiento normativo de un activo específico.
- **Data Producer (Productor de Datos):** Genera o introduce datos en el ecosistema.
- **Data Consumer (Consumidor de Datos):** Accede y utiliza los datos para análisis y soporte de procesos.
- **Data Custodian (Custodio de Datos):** Se encarga de la gestión técnica, almacenamiento y seguridad operativa.
- **Data Governance Manager (Gerente de Gobierno de Datos):** Lidera y coordina las actividades del programa para asegurar la aplicación de políticas y estándares.

## 6.9 Sistemas ERP y el Ecosistema Odoo

Los sistemas de **Planificación de Recursos Empresariales (ERP)** actúan como la columna vertebral operativa de una organización, integrando procesos que anteriormente residen en silos aislados. Odoo destaca por su arquitectura modular basada en una pila tecnológica moderna (Python y PostgreSQL), lo que permite una integración entre módulos clave como:

- **Ventas y CRM:** Control de ventas y relación con clientes.
- **Gestión de Inventarios:** Seguimiento en tiempo real.
- **Finanzas y Contabilidad:** Automatización de asientos contables.

En el ecosistema moderno, el ERP es la fuente de datos más crítica para el análisis de negocio, y el modelado dimensional es capaz de extraer datos de sus tablas normalizadas para transformarlos en estructuras de estrella que facilitan la visibilidad ejecutiva sobre la cadena de suministro y la rentabilidad.

# Capítulo 7

# Procedimiento y descripción de las actividades realizadas

A continuación, se describen las actividades realizadas durante el desarrollo del proyecto, así como su contribución al logro de los objetivos.

## 7.1 Levantamiento de requerimientos del área de Finanzas

Se realizan reuniones con el área de Finanzas con el objetivo de identificar sus necesidades de información, incluyendo los reportes requeridos, los indicadores clave y la periodicidad de actualización de los mismos. Como parte de este proceso, se analiza el procedimiento actual de extracción manual de datos desde el sistema ERP, así como el formato utilizado por el departamento financiero para la consolidación de la información.

### 7.1.1 Requerimientos de Información Financiera y Contable

Se establecen los siguientes requerimientos para la solución tecnológica:

1. **Automatización Integral de la Cédula de Gastos:** Se requiere la automatización completa de la cédula de gastos para eliminar la dependencia operativa de archivos de Excel complejos.

2. **Consolidación Transaccional (Odoo y COI) y Modelado Dimensional:** Es imperativo consolidar la información de egresos proveniente de múltiples orígenes, unificando los datos del ERP principal (Odoo) con los sistemas contables adicionales.

3. **Desarrollo de Vistas de Detalle para Validación de Integridad:** Se debe habilitar una vista analítica a nivel de transacción detallada que permita auditar la asignación de gastos.

4. **Desarrollo de Vistas Agregadas para Conciliación de Saldos:** Se requiere una vista con la información agregada a nivel de cuenta de gasto para permitir una confrontación y cuadre directo contra la balanza de comprobación.

## 7.2 Diseño de arquitectura del sistema

Se diseña la arquitectura tecnológica del sistema con el objetivo de establecer una solución que permita la extracción, procesamiento, almacenamiento y visualización automatizada de los datos financieros. Se define una infraestructura cloud en AWS como entorno principal de ejecución.

### 7.2.1 Diseño de arquitectura Data Lakehouse

El diseño se estructura bajo un modelo de Data Lakehouse fundamentado en tres pilares: robustez, flexibilidad y confiabilidad. Para el procesamiento de datos, se emplea Cómputo Elástico mediante instancias de Amazon EC2. Asimismo, se implementa un esquema de Almacenamiento Híbrido en el cual se provisionan volúmenes EBS para el Data Warehouse y Amazon S3 para el Data Lake. Finalmente, para garantizar la Seguridad y Gestión de Accesos (IAM), se asignan Roles IAM y Grupos de Seguridad que permiten el control del tráfico y los permisos correspondientes.

## 7.3 Creación y configuración de instancia EC2 en AWS

### 7.3.1 Creación de EC2

Se crea una instancia de Amazon EC2 en la plataforma de AWS, la cual funciona como servidor principal para alojar el motor de base de datos y orquestar los procesos de extracción, transformación y carga (ETL). Durante esta creación, se asignan los recursos de cómputo y memoria adecuados (4 vCPUs y 32 GB de RAM) para asegurar el correcto procesamiento y almacenamiento de la información financiera.

Para iniciar este proceso, se accede a la consola de administración mediante una cuenta de AWS, ya sea en la capa gratuita o de pago (ver figura 7.1).

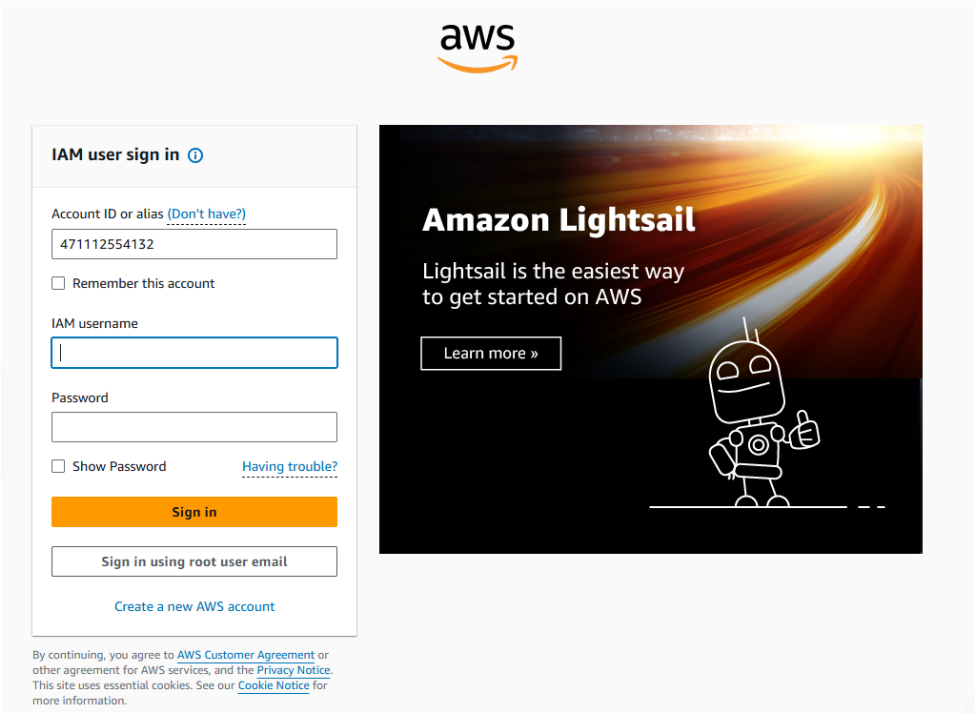


**Figura 7.1.** Inicio de sesión en la consola de AWS.

Posteriormente, en la barra de búsqueda principal se ingresa el término “EC2” y se selecciona el servicio correspondiente (ver figura 7.2).

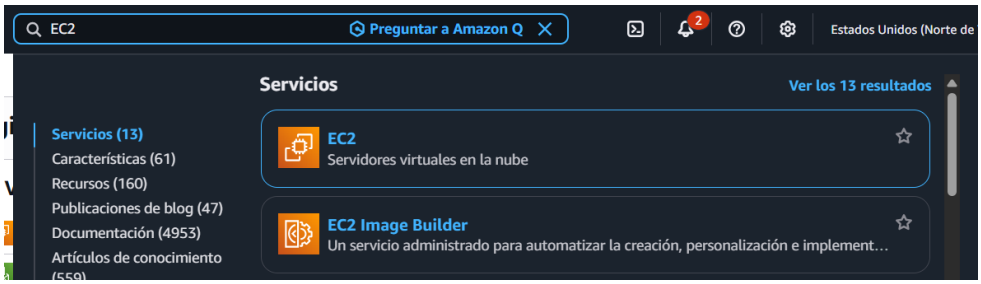


**Figura 7.2.** Búsqueda del servicio EC2 en la consola de AWS.

En el panel principal del servicio, se ubica y selecciona la opción *Lanzar instancia* para comenzar con la configuración (ver figura 7.3).

**Figura 7.3.** Opción para lanzar una nueva instancia.

Se despliega un formulario para definir los parámetros de lanzamiento. En primer lugar, se asigna un nombre identificativo a la instancia (ver figura 7.4).

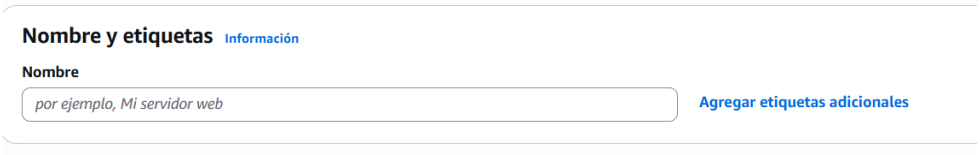


**Figura 7.4.** Asignación de nombre a la instancia EC2.

En la sección de Imágenes de Aplicaciones y Sistemas Operativos (AMI), se elige la distribución *Ubuntu* (ver figura 7.5), seleccionando específicamente la versión *Ubuntu Server 24.04 LTS* (ver figura 7.6).

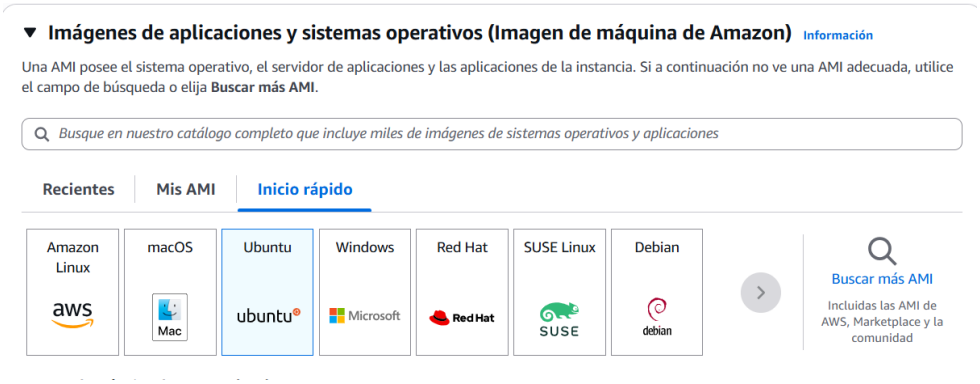


**Figura 7.5.** Selección de la imagen del sistema operativo Ubuntu.

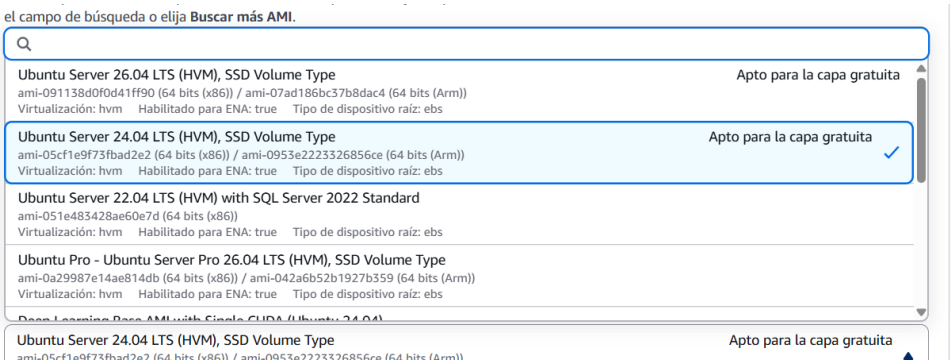


**Figura 7.6.** Selección de la versión Ubuntu Server 24.04.

Para el tipo de instancia, se decide por la familia `r7.xlarge` tras un análisis de costo-beneficio. Esta familia de instancias se encuentra optimizada para cargas de trabajo con requerimientos intensivos de memoria, alineándose con las necesidades de almacenamiento y procesamiento del motor de base de datos del proyecto (ver figura 7.7).

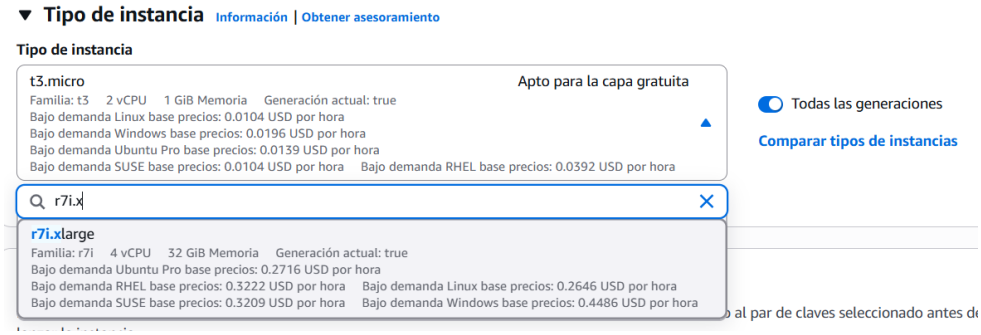


**Figura 7.7.** Selección del tipo de instancia optimizada para memoria.

A continuación, se genera un nuevo par de claves de seguridad, el cual resulta fundamental para establecer futuras conexiones seguras mediante el protocolo SSH (ver figura 7.8).

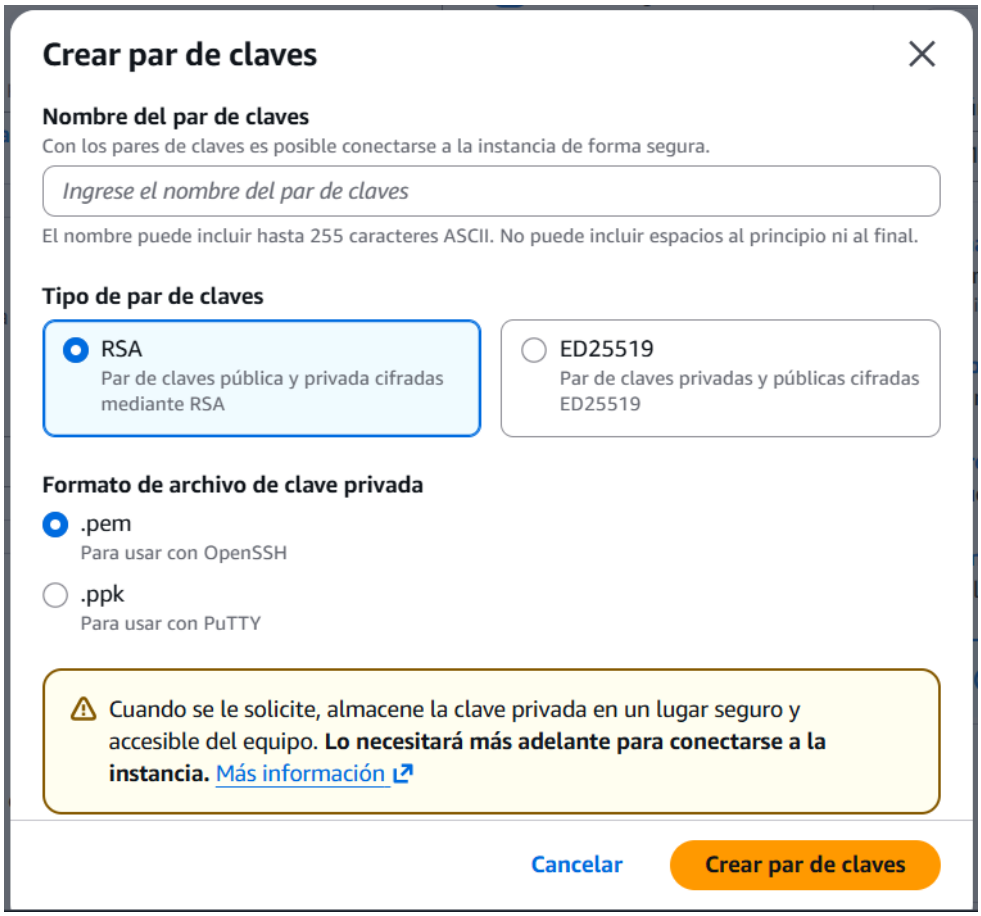


**Figura 7.8.** Creación del par de claves para conexión SSH.

En el apartado de *Configuraciones de red*, se despliegan las opciones avanzadas (ver figura 7.9). Se selecciona la misma VPC (*Virtual Private Cloud*) y subred donde operan las instancias de producción actuales. Esta estrategia evita los costos por transferencia externa de datos. Asimismo, se habilita la asignación automática de una dirección IP pública para permitir la posterior instalación de dependencias desde internet y se indica la creación de un nuevo grupo de seguridad que será editado más adelante (ver figura 7.10).

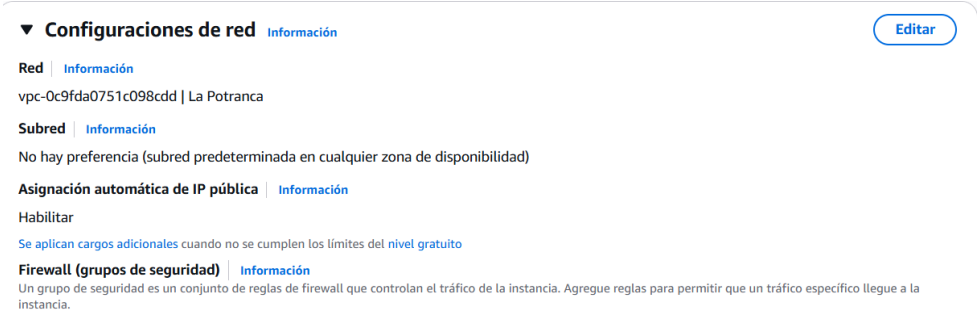


**Figura 7.9.** Edición de las configuraciones de red.

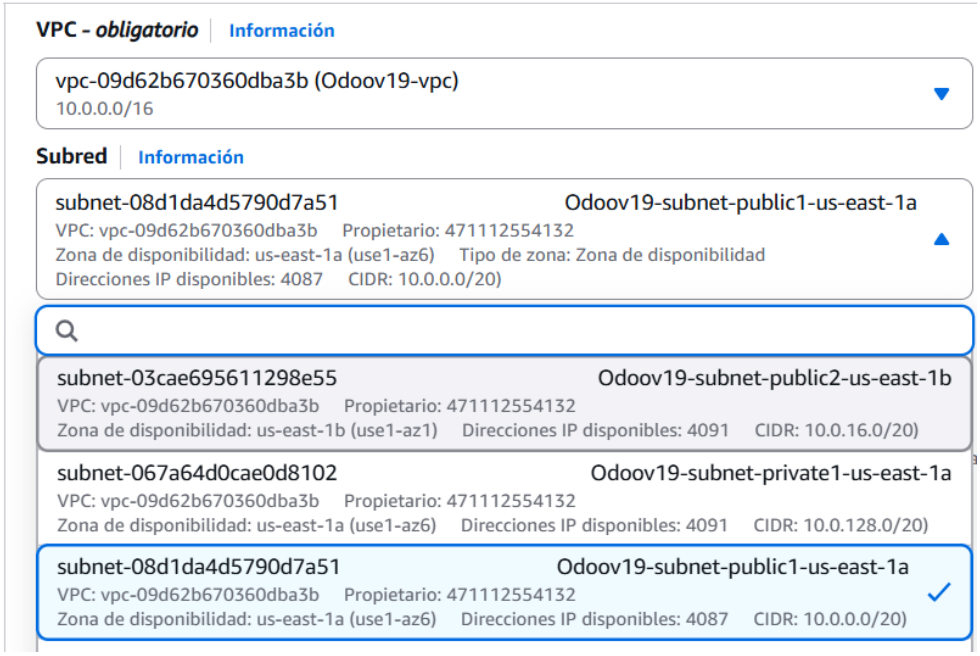


**Figura 7.10.** Configuración avanzada de red (VPC, subred e IP pública).

Finalmente, se configura el almacenamiento seleccionando la opción avanzada (ver figura 7.11). Se establece un volumen de tipo gp3 con una capacidad de 300 GB, configurado con 7000 IOPS y un rendimiento de 300 MB/s. Se especifica que el volumen no se elimine al terminar la instancia y se deshabilita el cifrado para esta capa de almacenamiento.

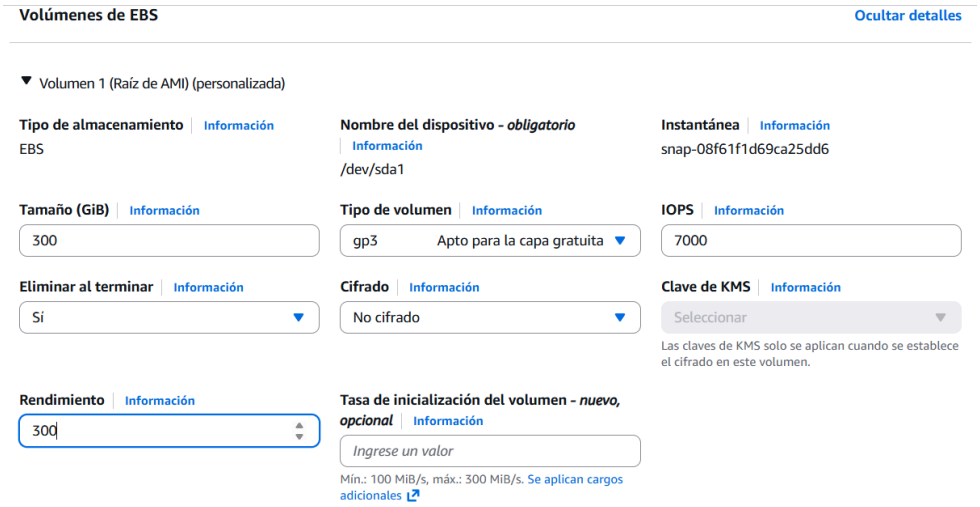


**Figura 7.11.** Configuración avanzada del volumen de almacenamiento EBS.

Para finalizar el proceso de configuración, se observa el resumen de los recursos seleccionados y se lanza la instancia (ver figura 7.12), lo cual inicia el despliegue de los recursos en la infraestructura cloud.

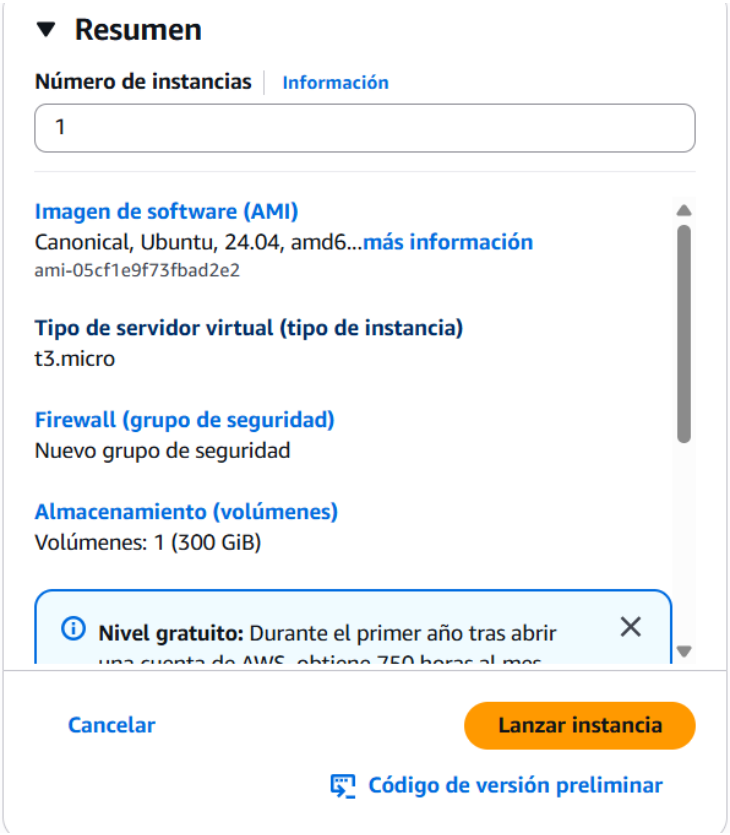


**Figura 7.12.** Resumen y lanzamiento de la instancia EC2.

### 7.3.2 Configuración de entorno Cloud (Linux)

Tras el despliegue de la instancia, se ejecuta de manera automática un script de inicialización (*User Data*) que prepara y configura el entorno operativo Linux. A continuación, se detalla el procedimiento ejecutado a través de los diversos comandos del script:

En primer lugar, se establece una variable de entorno para evitar interrupciones por diálogos interactivos durante la instalación, asegurando una ejecución desatendida. Seguidamente, se actualizan los repositorios y paquetes del sistema:

```
export DEBIAN_FRONTEND=noninteractive
echo "==> Iniciando User Data Script para DWH & Airflow (TEST
    VER)..."
apt update
apt upgrade -y
```

Posteriormente, se instalan las dependencias principales del sistema, que incluyen el sistema gestor de bases de datos PostgreSQL y las herramientas de desarrollo para Python necesarias para compilar librerías y configurar Apache Airflow:

```
apt install -y postgresql postgresql-contrib \
python3-pip python3-venv python3-dev \
libpq-dev build-essential libssl-dev libffi-dev
```

En la siguiente etapa, se configura PostgreSQL. Se modifica el archivo `pg_hba.conf` para permitir conexiones remotas y se insertan parámetros ajustados de rendimiento en el archivo `postgresql.conf`. Dichos parámetros optimizan el uso de memoria caché, memoria de trabajo, configuraciones de paralelismo y el comportamiento del *autovacuum* para la arquitectura de la instancia. Al finalizar, se reinicia el servicio:

```
PG_VERSION=$(ls /etc/postgresql | sort -V | tail -n 1)
echo "host all all 0.0.0.0/0 md5" >> /etc/postgresql/${
    PG_VERSION}/main/pg_hba.conf

cat >> /etc/postgresql/$PG_VERSION/main/postgresql.conf
Parametros optimizados para 2 vCPU y 8GB RAM
listen_addresses = '*'
shared_buffers = 2GB
work_mem = 32MB
```

```
max_worker_processes = 2
max_connections = 50
# (Entre otras configuraciones detalladas en el script original
    )
EOF_PG

systemctl restart postgresql
```

A continuación, se generan las bases de datos y los roles requeridos. Se crea la base de datos `airflow` junto con un usuario administrador específico, y se establece la base de datos `dlh_19` habilitando la extensión `pg_stat_statements` para monitoreo de consultas:

```
sudo -u postgres psql <<'EOF_SQL'
CREATE DATABASE airflow;
CREATE USER airflow_user WITH PASSWORD 'airflow';
ALTER ROLE airflow_user SET client_encoding TO 'utf8';
ALTER ROLE airflow_user SET default_transaction_isolation TO '
    read␣committed';
ALTER ROLE airflow_user SET timezone TO 'UTC';
GRANT ALL PRIVILEGES ON DATABASE airflow TO airflow_user;
\c airflow
GRANT ALL ON SCHEMA public TO airflow_user;
ALTER SCHEMA public OWNER TO airflow_user;
EOF_SQL

sudo -u postgres psql -c "CREATE␣DATABASE␣dlh_19;" || true
sudo -u postgres psql -d dlh_19 -c "CREATE␣EXTENSION␣IF␣NOT␣
    EXISTS␣pg_stat_statements;"
```

Para gestionar la ejecución de Apache Airflow, se crea un usuario de sistema denominado `airflow`. A través de este usuario, se configura un entorno virtual de Python y se instalan las librerías requeridas, así como la versión específica de Apache Airflow. También se definen las variables de entorno principales para la conexión a la base de datos:

```
id -u airflow &>/dev/null || useradd -m -s /bin/bash airflow

sudo -u airflow bash <<'EOF_AIRFLOW'
set -e
```

```
mkdir -p ~/airflow
cd ~/airflow
python3 -m venv venv-airflow
source venv-airflow/bin/activate
pip install --upgrade pip setuptools wheel
pip install psycopg2-binary asyncpg duckdb "sqlmesh[postgres]"

AIRFLOW_VERSION="3.0.1"
PYTHON_VERSION=$(python3 --version | cut -d "␣" -f 2 | cut -d "
    ." -f 1-2)
pip install "apache-airflow==${AIRFLOW_VERSION}" \
--constraint "https://raw.githubusercontent.com/apache/airflow/
    constraints-${AIRFLOW_VERSION}/constraints-${PYTHON_VERSION
    }.txt"

echo 'export␣AIRFLOW_HOME=~/airflow' >> ~/.bashrc
echo 'export␣AIRFLOW__DATABASE__SQL_ALCHEMY_CONN="postgresql+
    psycopg2://airflow_user:airflow@localhost:5432/airflow"' >>
    ~/.bashrc
EOF_AIRFLOW
```

Una vez instalado, se inicializa la base de datos interna de Airflow mediante un comando de migración y se crea el usuario administrador para el acceso a la interfaz de usuario web:

```
sudo -u airflow bash <<'EOF_DB'
set -e
cd ~/airflow
source venv-airflow/bin/activate
export AIRFLOW_HOME=~/airflow
export AIRFLOW__DATABASE__SQL_ALCHEMY_CONN="postgresql+psycopg2
    ://airflow_user:airflow@localhost:5432/airflow"
export AIRFLOW__CORE__EXECUTOR=LocalExecutor
airflow db migrate
airflow users create --username admin --firstname Admin --
    lastname Admin --role Admin --email admin@example.com --
    password admin || true
EOF_DB
```

Finalmente, para asegurar que los componentes de Apache Airflow se ejecuten de forma continua y se reinicien automáticamente en caso de falla o reinicio del servidor, se configuran tres servicios de `systemd` correspondientes a los procesos *Scheduler*, *DAG Processor* y *API Server*. Se habilitan y se inician para completar el despliegue del entorno:

```
# Creacion de archivos .service en /etc/systemd/system/
systemctl daemon-reload
systemctl enable airflow-scheduler airflow-dag-processor
    airflow-api-server
systemctl start airflow-scheduler airflow-dag-processor airflow
    -api-server
```

Una vez instaladas las herramientas y dependencias, se accede mediante un navegador web a la dirección IP del servidor en el puerto 8080 (`http://[ip_del_servidor]:8080`), donde se despliega el menú de inicio de sesión de Apache Airflow (ver figura 7.13).

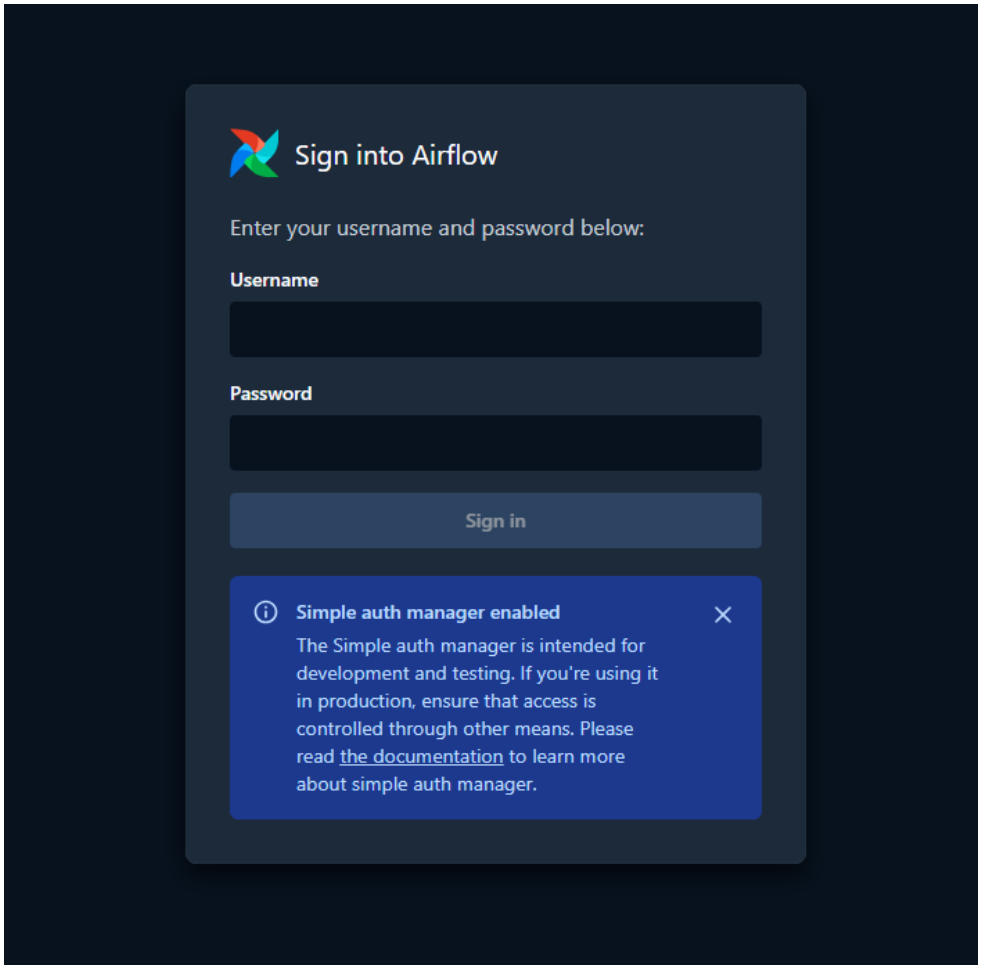


**Figura 7.13.** Pantalla de inicio de sesión de Apache Airflow.

Por defecto, el sistema crea un usuario administrador cuya contraseña autogenerada se encuentra alojada en el archivo `~/airflow/simple_auth_manager_passwords.json.generated`. Se extrae dicha credencial y se introduce en el formulario de acceso, lo cual permite ingresar a la consola principal de administración (ver figura 7.14).

**Figura 7.14.** Menú principal de la consola de administración de Apache Airflow.

Dentro de las configuraciones de administrador, se ubica el apartado de conexiones. Para la operación del presente proyecto se requiere establecer dos conexiones principales:

- Airflow → PostgreSQL (Data Lakehouse).
- Airflow → Base de datos de Producción.

Se procede a agregar cada una al seleccionar la opción *Add Connection*, lo que despliega el formulario correspondiente (ver figura 7.15).

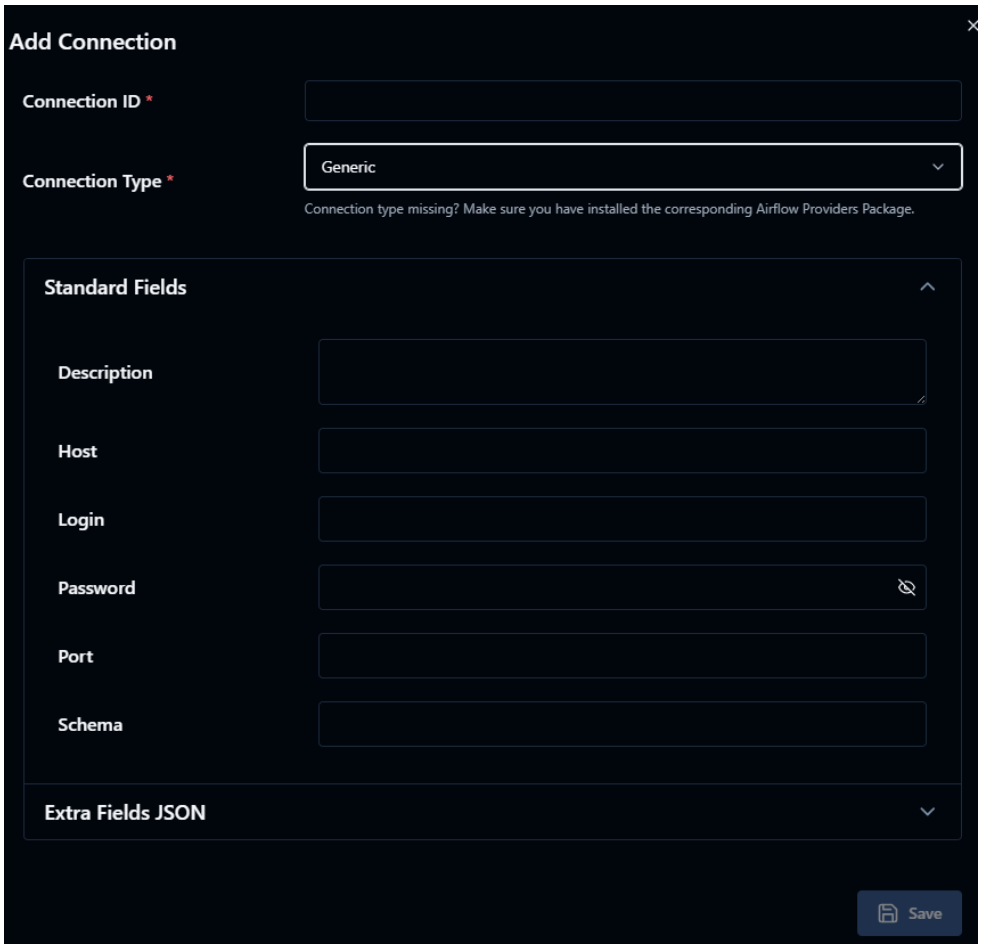


**Figura 7.15.** Menú para agregar una nueva conexión en Apache Airflow.

En dicho formulario se ingresan los siguientes datos solicitados y se guardan los cambios:

- **Host:** Dirección IP de la base de datos.
- **Login:** Usuario de la base de datos.
- **Password:** Contraseña de acceso.
- **Port:** Puerto de conexión.
- **Schema:** Nombre de la base de datos a la cual se accederá.

Para la visualización y control de los flujos de trabajo (DAGs), se ubica una pestaña denominada *DAGs* en la barra lateral de la interfaz. En esta sección se enlista la información correspondiente a cada uno de los procesos configurados (ver figura 7.16).

**Figura 7.16.** Vista general de los DAGs disponibles en el sistema.

Al ingresar a la información específica de un DAG, se observan principalmente dos elementos (ver figura 7.17):

1. Las tareas que componen el DAG y su estatus actual.
2. La información general y las métricas de las ejecuciones del DAG.

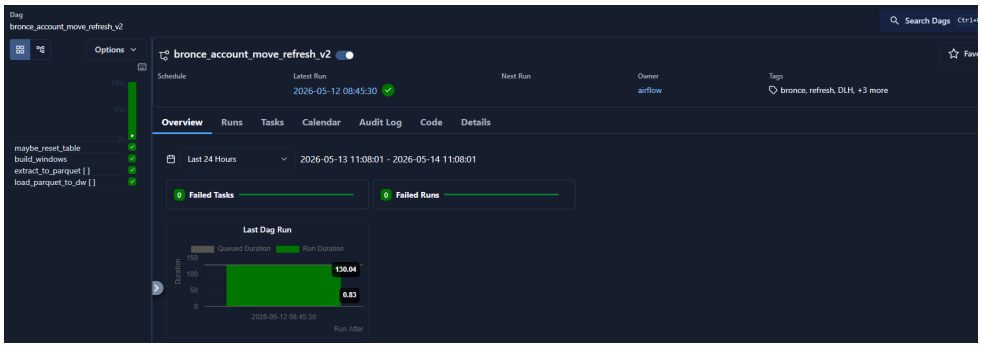


**Figura 7.17.** Información general y tareas de un DAG específico.

Asimismo, la plataforma permite visualizar el historial de las instancias de ejecución (DAG Runs) junto con su estado final, indicando si concluyeron de manera exitosa o fallida (ver figura 7.18).

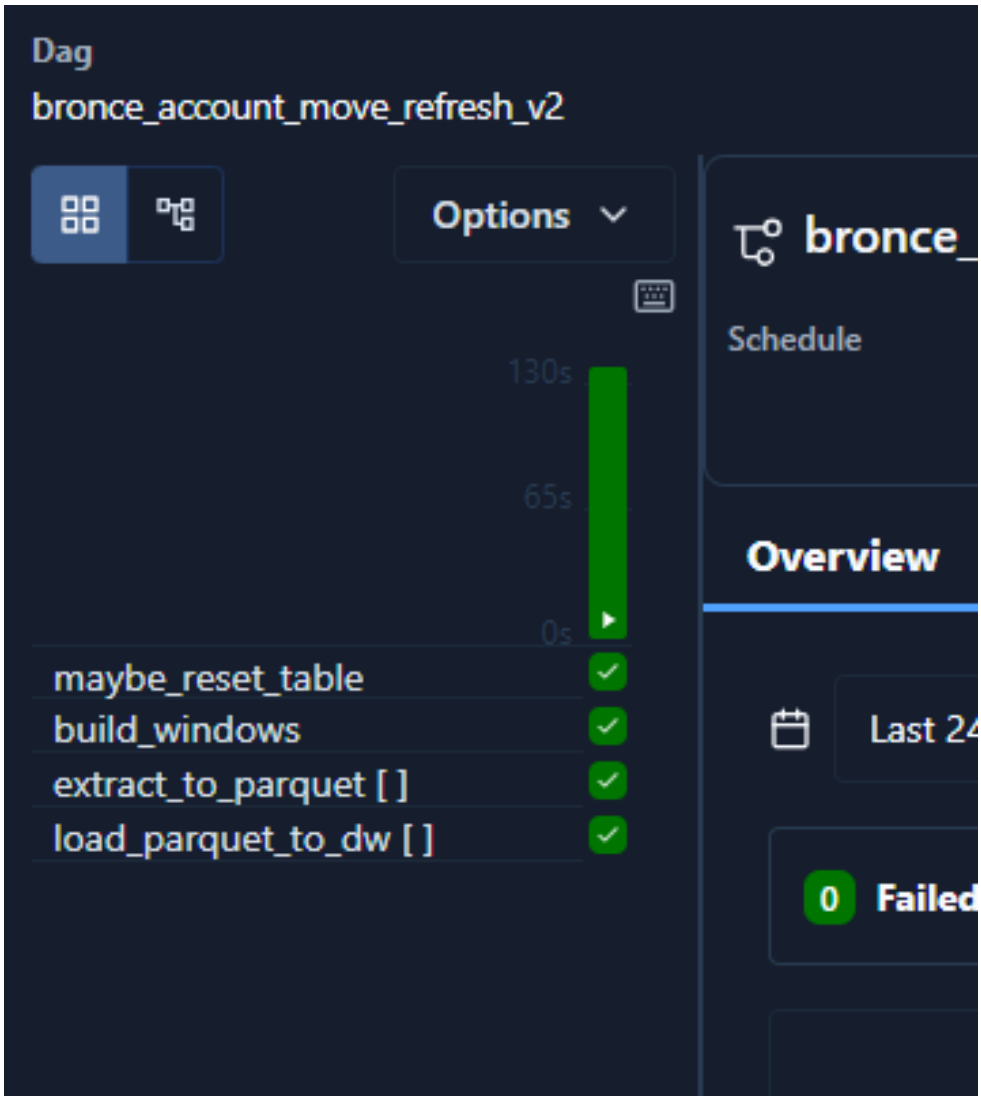


**Figura 7.18.** Historial e información de las instancias de ejecución del DAG.

Por último, se cuenta con una vista de calendario que facilita el monitoreo de las ejecuciones pasadas, su estado correspondiente y, en el caso de flujos programados recurrentemente, las ejecuciones planificadas a futuro (ver figura 7.19).

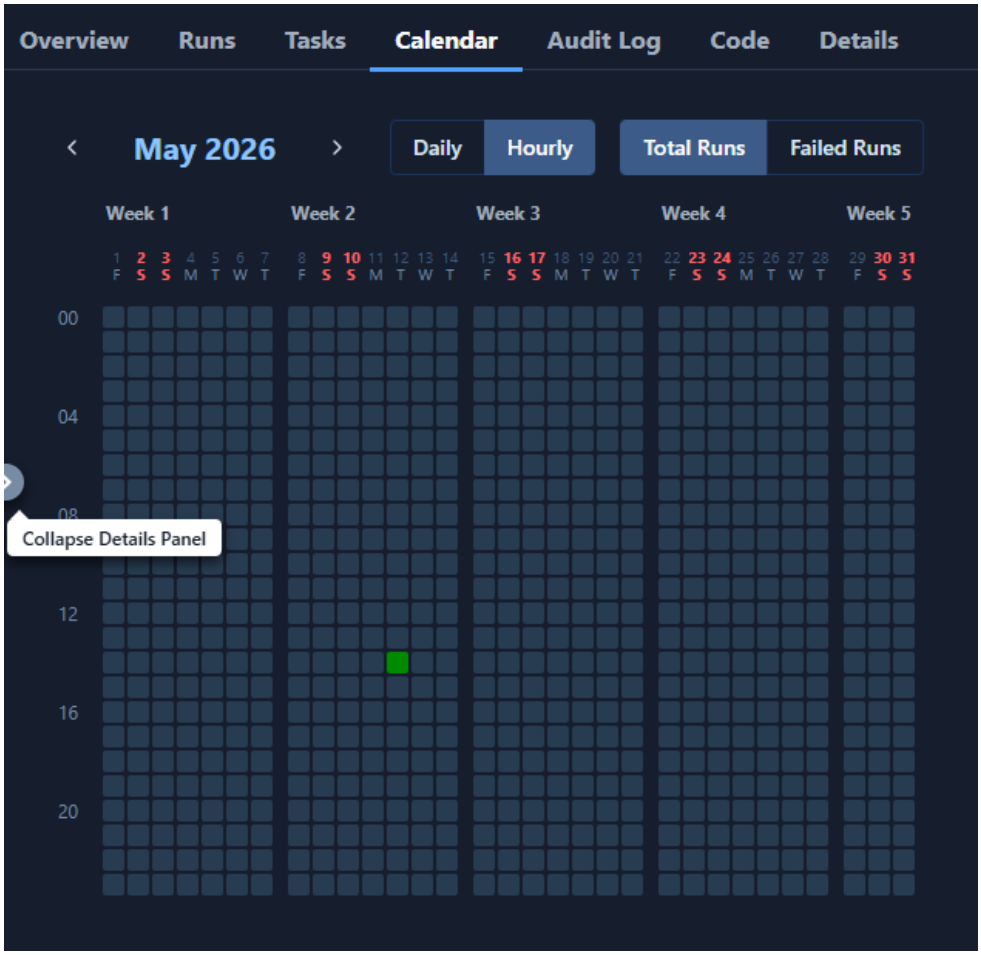


**Figura 7.19.** Vista de calendario con ejecuciones pasadas y futuras del DAG.

## 7.4 Creación de scripts ETL de datos con Airflow

Se desarrollan scripts en Python para implementar los procesos de extracción, transformación y carga de datos. Se utiliza Apache Airflow para definir flujos de ejecución mediante DAGs (*Directed Acyclic Graphs*).

### 7.4.1 Scripts de Extracción a Bronce

La Capa Bronce constituye la primera capa del Data Warehouse. Su objetivo consiste en almacenar una copia de los datos lo más cercana posible a la fuente original, sin aplicar transformaciones de negocio significativas. El flujo general de ingesta sigue tres etapas: extracción SQL a CSV, conversión de CSV a Parquet mediante DuckDB y carga masiva desde Parquet hacia el Data Warehouse en PostgreSQL.

A continuación se describe el proceso utilizando como ejemplo la ingesta de la tabla `account_move`, que corresponde a los movimientos contables y facturas del sistema ERP Odoo.

#### Inicio de la instancia de ingesta en Airflow

La ingesta se modela como un DAG de Apache Airflow. Para la tabla `account_move`, el DAG se identifica como `bronce_account_move_etl_v19-2`. Se configura sin calendario automático (`schedule=None`), con `catchup=False` y `max_active_runs=1`, lo que indica una ejecución controlada que evita múltiples corridas simultáneas del mismo DAG.

Dentro de la configuración, se generan ventanas mensuales que abarcan desde el año 2022 hasta marzo de 2026. Cada ventana contiene una fecha inicial y una fecha final, como se muestra a continuación:

```
{"start": "2025-01-01", "end": "2025-01-31"}
```

Esta estrategia de ventanas permite dividir la extracción en lotes manejables, evitando problemas de memoria o cargas excesivamente grandes.

#### Definición de la ventana de extracción

Cada instancia de ingesta recibe una ventana con los parámetros `start_date` y `end_date`. A partir de estos valores, se construyen las rutas temporales para almacenar los archivos intermedios del mes procesado:

```
start_date = window["start"]
end_date   = window["end"]
csv_path     = "/tmp/account_move_YYYY_MM.csv"
parquet_path = "/tmp/account_move_YYYY_MM.parquet"
```

Esta lógica se ejecuta dentro de la tarea `extract_to_parquet`.

### Lectura del SQL de extracción

La consulta de extracción se encuentra almacenada en el directorio `sql/bronce/account_move.sql`. El DAG lee este archivo como una plantilla SQL y reemplaza los parámetros `{{ start_date }}` y `{{ end_date }}` por las fechas reales de la ventana correspondiente.

La consulta filtra los registros por rango de fechas, procesando únicamente los movimientos del periodo asignado:

```
FROM account_move
WHERE "date" >= '{{ start_date }}'::date
  AND "date" <  '{{ end_date }}'::date
ORDER BY "date" ASC, id ASC;
```

### Conexión a la base de datos fuente

La conexión hacia la base de datos de producción de Odoo se obtiene desde Apache Airflow mediante la función:

```
BaseHook.get_connection("potranca19_prod")
```

Dicha conexión contiene los parámetros de *host*, puerto, usuario, contraseña y nombre de la base de datos del origen PostgreSQL.

### Extracción de datos a CSV

La extracción se realiza mediante el comando `psql` utilizando la directiva `\copy`, la cual envía el resultado de la consulta directamente hacia un archivo CSV local:

```
\copy ({query}) TO STDOUT WITH CSV HEADER
```

El DAG escribe el resultado directamente en disco (`stdout=csv_file`), lo cual resulta fundamental porque evita cargar la totalidad del resultado en memoria. Para tablas de gran volumen como `account_move`, este enfoque de transmisión directa (*streaming*) reduce significativamente el riesgo de saturar la memoria RAM.

### Validación de la extracción

Tras generar el archivo CSV, el DAG verifica si este contiene registros. Se calcula el número de filas restando la línea de encabezado:

```
row_count = sum(1 for _ in f) - 1
```

Si el mes no contiene datos, se elimina el archivo CSV y se devuelve un *payload* con `parquet_path: None`, indicando a la tarea siguiente que no debe realizar ninguna carga para esa ventana.

### Conversión de CSV a Parquet con DuckDB

Cuando el CSV contiene registros válidos, DuckDB lo convierte al formato Parquet:

```
COPY (
    SELECT *
    FROM read_csv_auto(...)
)
TO '{parquet_path}'
(FORMAT PARQUET, COMPRESSION 'ZSTD', ROW_GROUP_SIZE 100000);
```

En esta ingesta, el DAG fuerza todos los campos a `VARCHAR` para evitar conflictos de tipos durante la carga histórica. La resolución de tipos reales se delega a las capas posteriores, principalmente la Capa Plata. Esta decisión refuerza un principio clave de la Capa Bronce: no limpiar ni interpretar el dato en esta etapa, priorizando su recepción y conservación íntegra.

Una vez completada la conversión, se elimina el archivo CSV temporal para reducir el uso de almacenamiento:

```
if os.path.exists(csv_path):
    os.remove(csv_path)
```

La tarea devuelve un diccionario con las fechas de la ventana y la ruta del archivo Parquet generado, el cual se transfiere como *payload* a la tarea de carga.

### Preparación de la carga hacia el Data Warehouse

La siguiente tarea, denominada `load_parquet_to_dw`, recibe el *payload* anterior. Si el valor de `parquet_path` es `None`, la tarea concluye sin ejecutar la carga.

En caso contrario, se obtiene la conexión al Data Warehouse mediante:

```
BaseHook.get_connection("potranca19_DLH")
```

Con los parámetros obtenidos se construye un DSN (*Data Source Name*) de PostgreSQL.

### Conexión de DuckDB al Data Warehouse

Los scripts de carga utilizan la extensión `postgres` de DuckDB para conectarse directamente al Data Warehouse:

```
INSTALL postgres;
LOAD postgres;
ATTACH '{{ pg_dsn }}'
AS pg_dw (TYPE POSTGRES);
```

Esta configuración permite que DuckDB lea el archivo Parquet local y escriba directamente en PostgreSQL sin intermediarios adicionales.

### Creación del esquema y tabla destino

Antes de ejecutar la carga, el script crea el esquema Bronce si este no existe:

```
CREATE SCHEMA IF NOT EXISTS pg_dw.bronce;
```

Asimismo, se crea la tabla destino utilizando la estructura del archivo Parquet, sin insertar filas inicialmente:

```
CREATE TABLE IF NOT EXISTS pg_dw.bronce.account_move AS
SELECT * FROM read_parquet('{{ parquet_path }}')
WHERE 1=0;
```

### Aplicación de idempotencia y carga de datos

Para garantizar que la ingesta pueda repetirse sin generar duplicados, se aplica una lógica de refresco que elimina previamente los registros del rango a recargar:

```
DELETE FROM pg_dw.bronce.account_move
WHERE "date" >= '{{ start_date }}'
  AND "date" <  '{{ end_date }}';
```

Este paso resulta clave: si una instancia falla o se requiere reprocesar un mes, se eliminan primero los registros del rango correspondiente y luego se vuelven a insertar.

Finalmente, DuckDB carga el contenido del Parquet hacia PostgreSQL:

```
INSERT INTO pg_dw.bronce.account_move
SELECT * FROM read_parquet('{{ parquet_path }}');
```

#### Limpieza de archivos temporales

Tras ejecutar la carga, el DAG cierra la conexión de DuckDB y elimina el archivo Parquet temporal:

```
if os.path.exists(parquet_path):
    os.remove(parquet_path)
```

Esto deja la instancia en un estado limpio y evita la acumulación de archivos en el directorio temporal.

### 7.4.2 Ingesta de datos del Modelo de datos Bronce a Plata con SQLmesh

A diferencia de la Capa Bronce, donde se extrae información directamente desde el sistema origen, la Capa Plata se construye a partir de los datos ya existentes en las tablas `bronce.*` dentro de PostgreSQL. Para esta transformación se utiliza SQLMesh, un marco de trabajo que permite definir modelos SQL declarativos con control de versiones y materialización automatizada.

El flujo general se resume de la siguiente manera: SQLMesh lee las tablas crudas del esquema `bronce`, aplica transformaciones de estandarización y materializa los resultados en el esquema `plata`.

#### Ejecución del plan de SQLMesh desde Airflow

El DAG encargado de iniciar el proceso se denomina `sqlmesh_plan`. Dentro de este, un `BashOperator` activa el entorno virtual de Python, se posiciona en el directorio `sql/` y ejecuta el siguiente comando:

```
sqlmesh plan prod --auto-apply
```

Este comando indica a SQLMesh que debe calcular el plan de cambios pendientes y aplicarlo automáticamente en el ambiente de producción (`prod`). El DAG inyecta las credenciales del Data Warehouse mediante variables de entorno (`PG_HOST`, `PG_USER`, `PG_PASSWORD`, `PG_DATABASE` y `PG_PORT`), las cuales se obtienen de la conexión de Airflow `potranca19_DLH`.

### Configuración de conexión de SQLMesh

La configuración de SQLMesh se encuentra en el archivo `sql/config.py`, donde se define el *gateway* principal denominado `potranca19_DLH` y la conexión PostgreSQL que utiliza las variables de entorno previamente inyectadas. También se establece que el dialecto por defecto de los modelos es PostgreSQL, lo que significa que todas las transformaciones se compilan y ejecutan como sentencias SQL nativas del motor del Data Warehouse.

### Descubrimiento y estructura de los modelos Plata

Los modelos de la Capa Plata se ubican en el directorio `sql/models/plata/` y siguen la convención de nomenclatura `plata__<entidad>.sql`. Cada archivo inicia con un bloque `MODEL` que declara las propiedades del modelo. Por ejemplo, para la tabla `account_move`:

```
MODEL (
    name plata.account_move,
    kind FULL,
    grain (id),
    description 'Carga bronce.account_move -> plata.
        account_move
                con casteos y valores NOT NULL'
);
```

La propiedad `kind FULL` indica que el modelo se reconstruye por completo en cada ejecución, generando una regeneración integral de la tabla Plata con base en el estado actual de Bronce. La propiedad `grain` define la granularidad esperada del modelo, es decir, un registro por identificador.

### Transformaciones aplicadas en la Capa Plata

Después del bloque `MODEL`, cada archivo contiene la sentencia `SELECT` que define la transformación. La fuente de datos siempre es una tabla del esquema `bronce`. Las transformaciones que se aplican de manera general incluyen:

1. **Casteo de tipos:** Cada columna se convierte explícitamente al tipo de dato correcto. Los campos numéricos se transforman a `numeric` o `int4`, las fechas a `date` y los textos a `varchar`:

   ```
   CAST(b.id AS int4) AS id,
   b.amount_total::numeric AS amount_total,
   b.date::date AS "date",
   ```

2. **Protección de campos obligatorios:** Se utiliza la función `COALESCE` para asignar valores predeterminados a campos críticos que no deben contener nulos:

   ```
   COALESCE(b.date::date, CURRENT_DATE) AS "date",
   COALESCE(b.state::varchar, 'draft') AS state,
   COALESCE(b.journal_id::int4, 0) AS journal_id,
   ```

3. **Conversión de estructuras JSONB:** Los campos que provienen de Odoo como estructuras JSON se convierten explícitamente al tipo `jsonb` de PostgreSQL, permitiendo que las capas posteriores los consulten de forma nativa:

   ```
   b.sending_data::jsonb AS sending_data,
   b.analytic_distribution::jsonb AS analytic_distribution,
   ```

4. **Reglas de integridad de negocio:** Se implementan validaciones específicas según la entidad. Por ejemplo, en la tabla `account_move_line`, se evita la asignación de una cuenta contable a líneas que representan secciones o notas:

   ```
   CASE
       WHEN b.display_type IN ('line_section','line_note')
       THEN NULL
       ELSE b.account_id::int4
   END AS account_id
   ```

   Asimismo, se protege la consistencia entre débito y crédito para evitar que una misma línea contenga valores simultáneos en ambos campos:

```
CASE WHEN b.credit::numeric > 0 THEN 0
      ELSE b.debit::numeric END AS debit,
CASE WHEN b.debit::numeric > 0 THEN 0
      ELSE b.credit::numeric END AS credit,
```

#### Materialización y disponibilidad para la Capa Oro

Al ejecutar `sqlmesh plan prod --auto-apply`, SQLMesh determina qué modelos requieren actualización, ejecuta las transformaciones directamente dentro del motor PostgreSQL del Data Warehouse y materializa los resultados en el esquema `plata`. Conceptualmente, para cada modelo de tipo `FULL`, el proceso equivale a reconstruir la tabla completa a partir del `SELECT` definido.

Una vez completada la materialización, las tablas del esquema `plata` (como `plata.account_move`, `plata.account_move_line`, `plata.account_account` y `plata.res_partner`) quedan tipadas, limpias y normalizadas, listas para ser consumidas por los modelos de la Capa Oro, donde se construyen las tablas de hechos y dimensiones del modelo dimensional.

### 7.4.3 Construcción de Modelo dimensional y tablas de hechos

La Capa Oro constituye la capa analítica del proyecto. A diferencia de la Capa Plata, donde el objetivo principal es la estandarización de tipos y limpieza básica, en la Capa Oro se construyen modelos orientados a negocio bajo un enfoque de esquema estrella, compuesto por tablas de hechos y dimensiones. Al igual que la Capa Plata, esta transformación se ejecuta mediante SQLMesh a través del mismo DAG `sqlmesh_plan`, por lo que los modelos de ambas capas se procesan en una misma ejecución del comando `sqlmesh plan prod --auto-apply`.

#### Tabla de hechos: `oro.fact_gastos_distribucion`

El modelo central de la Capa Oro es `oro.fact_gastos_distribucion`, una tabla de hechos financiera que toma las líneas contables estandarizadas desde Plata, expande la distribución analítica de Odoo y deja la información lista para el análisis de gastos por cuenta, compañía, proveedor, usuario y cuenta analítica. Su declaración es la siguiente:

```
MODEL (
    name oro.fact_gastos_distribucion,
```

```
    kind FULL,
    grain (aml_id),
    description 'Fact de distribucion analitica de gastos'
);
```

La granularidad declarada es una fila por `aml_id`, que corresponde a una línea de `account_move_line`. El modelo se reconstruye por completo en cada ejecución (`kind FULL`).

### Flujo de construcción del modelo

El modelo se construye mediante una serie de CTEs (*Common Table Expressions*) que procesan los datos de forma secuencial:

1. **Lectura de líneas contables:** Se parte de `plata.account_move_line`, que ya contiene campos tipados y limpios como `id`, `move_id`, `account_id`, `debit`, `credit`, `balance` y `analytic_distribution`.

2. **Expansión de la distribución analítica:** La CTE `distribucion_expandida` toma el campo JSONB `analytic_distribution` y lo convierte en pares clave-valor mediante `CROSS JOIN LATERAL jsonb_each_text(...)`. Se contempla un caso especial donde el JSONB puede llegar como tipo *string* envuelto, el cual se desempaqueta automáticamente. Solo se incluyen movimientos con estado publicado (`parent_state = 'posted'`).

3. **Separación de claves compuestas:** La CTE `distribucion_individual` maneja las claves analíticas compuestas de Odoo (por ejemplo, "10,25") separándolas mediante `unnest (string_to_array(...))` y convirtiendo el porcentaje analítico a tipo numérico.

4. **Conservación de líneas sin analítica:** La CTE `sin_analitica` captura las líneas contables publicadas que no poseen distribución analítica, asignando valores nulos a los campos correspondientes. Esto permite analizar gastos incluso cuando carecen de clasificación analítica.

5. **Unión de conjuntos:** La CTE `union_data` combina ambos conjuntos mediante `UNION ALL`, conformando un universo completo de líneas contables.

6. **Clasificación y pivoteo:** La CTE `pivoteada` se une con la dimensión `oro.dim_analytic` para clasificar cada cuenta analítica y determinar si pertenece a un plan de tipo *Former Tag*. Se aplica una agrupación a nivel de línea contable (`aml_id`), separando la cuenta analítica principal (`analytic_account_id`) de la etiqueta secundaria (`former_tag_id`).

7. **Enriquecimiento final:** En el `SELECT` final, se obtiene el usuario responsable del movimiento desde `plata.account_move` y se filtra por cuentas contables de gasto relevantes (códigos `601.*`, `899.01.200` y `899.01.100`) mediante un `JOIN` con `oro.dim_account`.

Los campos finales del modelo incluyen las llaves dimensionales (`company_id`, `partner_id`, `account_id`, `user_id`, `analytic_account_id`, `former_tag_id`), las medidas financieras (`debit`, `credit`, `balance`) y una bandera de segmentación (`is_sin_analitica`) que indica si la línea carece de clasificación analítica.

### Tablas de dimensiones

El modelo dimensional se complementa con las siguientes tablas de dimensiones, cada una construida desde las tablas estandarizadas de la Capa Plata:

- `oro.dim_account`**:** Se construye desde `plata.account_account`. Contiene el identificador de cuenta, nombre, código contable (almacenado como JSONB para soportar variaciones por compañía) y tipo de cuenta. Permite responder preguntas sobre la clasificación contable del gasto.
- `oro.dim_analytic`**:** Se construye desde `plata.account_analytic_account` y `plata.account_anal`
  Incluye el nombre y código de la cuenta analítica, el plan analítico asociado y la bandera `is_former_tag`. Permite analizar gastos por estructura analítica y clasificación organizacional.
- `oro.dim_company`**:** Se construye desde `plata.res_company` y `plata.res_partner`. Contiene el nombre de la empresa, RFC, moneda y país fiscal. Permite segmentar gastos por entidad legal o razón social.
- `oro.dim_partner`**:** Se construye desde `plata.res_partner`, enriquecida con información del socio comercial y la compañía asociada. Incluye nombre completo, RFC, correo electrónico, ciudad y rangos de cliente/proveedor. Permite analizar gastos por proveedor, cliente o entidad comercial.
- `oro.dim_users`**:** Se construye desde `plata.res_users` y `plata.res_partner`. Contiene el nombre del usuario, región, sucursal, tipo de usuario y estado activo. Permite asociar cada gasto al usuario responsable del movimiento.

**Esquema estrella resultante**

El modelo dimensional resultante sigue un enfoque de esquema estrella donde la tabla central `oro.fact_gastos_distribucion` se relaciona con las dimensiones de negocio a través de sus llaves foráneas. Esta estructura permite realizar consultas analíticas como: el total de gasto por cuenta contable, la distribución de gastos por empresa, la identificación de proveedores con mayor concentración de gasto, el análisis por usuario responsable y la segmentación por estructura analítica. La información queda preparada para ser consumida directamente por herramientas de visualización como Power BI.

## 7.5 Configuración de EC2 para exponer datos a Power BI

Se configura la instancia EC2 para permitir el acceso seguro de Power BI a los datos procesados, estableciendo una integración directa entre la infraestructura cloud y la herramienta de visualización.

### 7.5.1 Configuración de Ec2 para exposición de datos

La exposición de los datos procesados desde la instancia EC2 hacia las herramientas de visualización se gestiona mediante mecanismos de seguridad en dos niveles: infraestructura y base de datos. A nivel de infraestructura, se establece una regla específica en el Grupo de Seguridad de AWS que permite el tráfico entrante únicamente desde las direcciones IP autorizadas por la organización.

En el motor de base de datos PostgreSQL, se crea un usuario exclusivo para la plataforma de Business Intelligence (BI). Este usuario se configura con permisos restringidos, permitiendo el acceso únicamente a los elementos que conforman la Capa Oro. Esta estrategia de seguridad asegura que la herramienta de visualización solo tenga visibilidad sobre los modelos dimensionales y tablas de hechos finales, protegiendo la integridad de los datos crudos y estandarizados presentes en las capas Bronce y Plata.

### 7.5.2 Configuración de Power BI service

Una vez establecida la conectividad con la instancia EC2, se procede a la configuración en el entorno local mediante Power BI Desktop utilizando un conector ODBC (*Open Database*

*Connectivity*). Se proporcionan las credenciales del usuario exclusivo y se seleccionan las tablas de hechos y dimensiones de la Capa Oro necesarias para el desarrollo del tablero de control.

Al realizar la importación de los datos, Power BI genera un objeto técnico denominado Modelo Semántico. Sobre este modelo se definen las relaciones, jerarquías y medidas necesarias para la analítica. Una vez finalizado el desarrollo del tablero, el proyecto se publica en el espacio de trabajo (*Workspace*) seleccionado dentro de la nube de Microsoft.

Posteriormente, se accede a la consola de Power BI Service para localizar el Modelo Semántico publicado. En este entorno, se configuran las opciones de actualización programada de datos y se establece la conexión persistente entre el servicio en la nube y el Data Lakehouse alojado en AWS, garantizando que los reportes reflejen la información más reciente procesada por el pipeline de datos.

# Capítulo 8

# Resultados

Se presentan los resultados obtenidos tras la implementación de la arquitectura de datos y el desarrollo de los tableros de visualización.

## 8.1 Dashboards de Inteligencia de Negocios

Para satisfacer las necesidades de información financiera y control operativo, se implementan tres vistas analíticas principales:

1. **Matriz Jerárquica Interactiva:** Permite la visualización consolidada de la información de gastos desde el nivel general hasta el detalle granular por departamento o cuenta específica.
2. **Panel Tabular de Nivel Atómico:** Expone cada transacción individual registrada, integrando folios de movimientos, fechas y referencias cruzadas para auditorías precisas.
3. **Vista de Confrontación Contable:** Presenta un resumen totalizado de los montos agrupados por cuenta contable y un desglose por fecha para el cuadre contra la balanza de comprobación.

**Figura 8.1.** Matriz jerárquica interactiva para consolidación de gastos.

**Figura 8.2.** Panel tabular para validación de transacciones.

**Figura 8.3.** Vista de Confrontación Contable.

# Capítulo 9

# Conclusiones, recomendaciones y experiencia profesional adquirida

## 9.1 Conclusiones

Se concluye que la implementación de una arquitectura tecnológica basada en un modelo de Data Lakehouse reduce significativamente el tiempo de procesamiento de datos y mejora la confiabilidad de los reportes financieros. La adopción de una estrategia por capas (Bronce, Plata y Oro) permite separar de forma clara las responsabilidades de ingesta, estandarización y modelado analítico, lo cual facilita tanto el mantenimiento como la escalabilidad de la solución.

La automatización de los procesos ETL mediante Apache Airflow elimina la dependencia de procedimientos manuales basados en archivos de Excel, los cuales resultan propensos a errores y demandan un alto consumo de tiempo por parte del personal del área de Finanzas. Con la solución implementada, la consolidación de la cédula de gastos se realiza de forma programada y repetible, garantizando la consistencia de la información en cada ejecución.

El uso de SQLMesh como marco de transformación declarativa permite versionar y reproducir cada modelo de datos, asegurando la trazabilidad de las transformaciones aplicadas desde los datos crudos hasta las tablas de hechos y dimensiones. Este enfoque refuerza la confiabilidad del proceso al hacer explícitas las reglas de negocio contenidas en cada capa.

La integración entre la infraestructura cloud de AWS y Power BI Service posibilita la actualización automatizada de los tableros de control, proporcionando al área financiera acceso a información consolidada y actualizada sin intervención manual. La validación por parte del área de Finanzas confirma que la solución cumple con los requerimientos establecidos en cuanto a disponibilidad, granularidad y oportunidad de la información.

## 9.2 Recomendaciones

Se recomienda incorporar una capa de validación de calidad de datos mediante herramientas como Soda Core, con el objetivo de detectar anomalías o registros inconsistentes antes de que estos alcancen las capas de transformación. Esta medida contribuye a fortalecer la integridad de la información procesada.

Asimismo, se sugiere evaluar la implementación de un esquema de actualización incremental en las tablas de mayor volumen, lo cual permite reducir los tiempos de procesamiento y optimizar el uso de recursos de cómputo. De igual manera, se recomienda documentar las reglas de negocio aplicadas en cada modelo de la Capa Oro, de modo que cualquier modificación futura pueda realizarse con pleno conocimiento de la lógica existente.

En materia de gobernanza y calidad de datos, se recomienda la adopción de OpenMetadata como plataforma centralizada de catálogo de datos. Esta herramienta permite registrar de forma automática los metadatos de cada tabla y modelo del Data Lakehouse, establecer políticas de propiedad y clasificación de activos de datos, y ejecutar pruebas de calidad programadas que validen la completitud, unicidad y frescura de la información en cada capa. Su integración con el pipeline existente facilita la visibilidad del linaje de datos desde la fuente hasta los tableros de control, fortaleciendo la trazabilidad y el cumplimiento de estándares de calidad dentro de la organización.

Adicionalmente, se recomienda evolucionar las tablas de dimensiones del modelo estrella mediante la implementación de estrategias de Slowly Changing Dimensions (SCD). Para dimensiones donde únicamente se requiere conservar el valor más reciente, como el nombre o correo electrónico de un proveedor, resulta apropiado aplicar SCD Tipo 1, el cual sobrescribe el registro anterior. Para dimensiones cuyo historial es relevante para el análisis, como cambios en la clasificación contable o en la estructura analítica, se recomienda SCD Tipo 2, el cual preserva cada versión del registro mediante campos de vigencia (fecha de inicio y fecha de fin) y una bandera de registro activo. Finalmente, para casos donde se requiere conservar únicamente el valor anterior y el actual sin mantener un historial completo, se sugiere evaluar SCD Tipo 3 mediante la adición de columnas adicionales que almacenen el valor previo. Esta estrategia permite enriquecer la capacidad analítica del modelo dimensional al habilitar comparaciones históricas y auditorías de cambios en las entidades de negocio.

Finalmente, se considera conveniente explorar la migración hacia un servicio de base de datos administrado, como Amazon RDS, para reducir la carga operativa asociada a la administración directa del motor PostgreSQL sobre la instancia EC2.

## 9.3 Experiencia profesional adquirida

Durante el desarrollo del proyecto se adquiere experiencia profesional en diversas áreas de la ingeniería de datos. Se fortalece el dominio en el diseño y despliegue de infraestructura cloud mediante Amazon Web Services, incluyendo la configuración de instancias EC2, volúmenes EBS, almacenamiento S3 y la gestión de accesos a través de IAM.

Se desarrolla competencia práctica en la orquestación de flujos de datos con Apache Airflow, abarcando la definición de DAGs, la gestión de conexiones, la configuración de servicios de sistema y la resolución de incidencias en ambientes de producción. Asimismo, se profundiza en el uso de SQLMesh para la construcción de pipelines de transformación declarativos con control de versiones y materialización automatizada.

En el ámbito del modelado de datos, se adquiere experiencia en el diseño de esquemas estrella compuestos por tablas de hechos y dimensiones, orientados al análisis financiero. Se refuerza el manejo de PostgreSQL en entornos analíticos, incluyendo la optimización de parámetros de rendimiento y la administración de esquemas.

Por último, se fortalece la capacidad de comunicación con áreas no técnicas, al traducir los requerimientos del departamento de Finanzas en soluciones tecnológicas concretas y validar los resultados de manera conjunta. Esta interacción contribuye a desarrollar una visión integral que combina el conocimiento técnico con la comprensión de las necesidades de negocio.

# Capítulo 10

# Competencias desarrolladas y/o aplicadas

Durante el desarrollo del proyecto de residencia profesional se aplican diversas competencias profesionales, destacando aquellas relacionadas con la ingeniería de datos. En primer lugar, se demuestra capacidad técnica para el diseño y creación de pipelines de datos automatizados, garantizando una correcta integración y procesamiento de la información. Asimismo, se desarrolla la habilidad para analizar requerimientos y traducirlos mediante el modelado de datos, estructurando esquemas analíticos que resuelven problemas específicos de la organización.

Por otro lado, se fortalece el trabajo en equipo y la comunicación efectiva con diferentes áreas de la empresa, asegurando la alineación de los objetivos técnicos con las necesidades operativas. Finalmente, se aplica el pensamiento crítico para la toma de decisiones en el diseño de las transformaciones de datos y el aseguramiento de la calidad de la información.

# Apéndice A

# Anexos

## A.1 Carta de autorización de la empresa

(Espacio para incluir PDF de la carta de autorización)